PROJET AWD · ÉQUIPE KEYSTONE · 4DAS_I4

MediVault

Votre santé, en sécurité, dans un seul dossier.

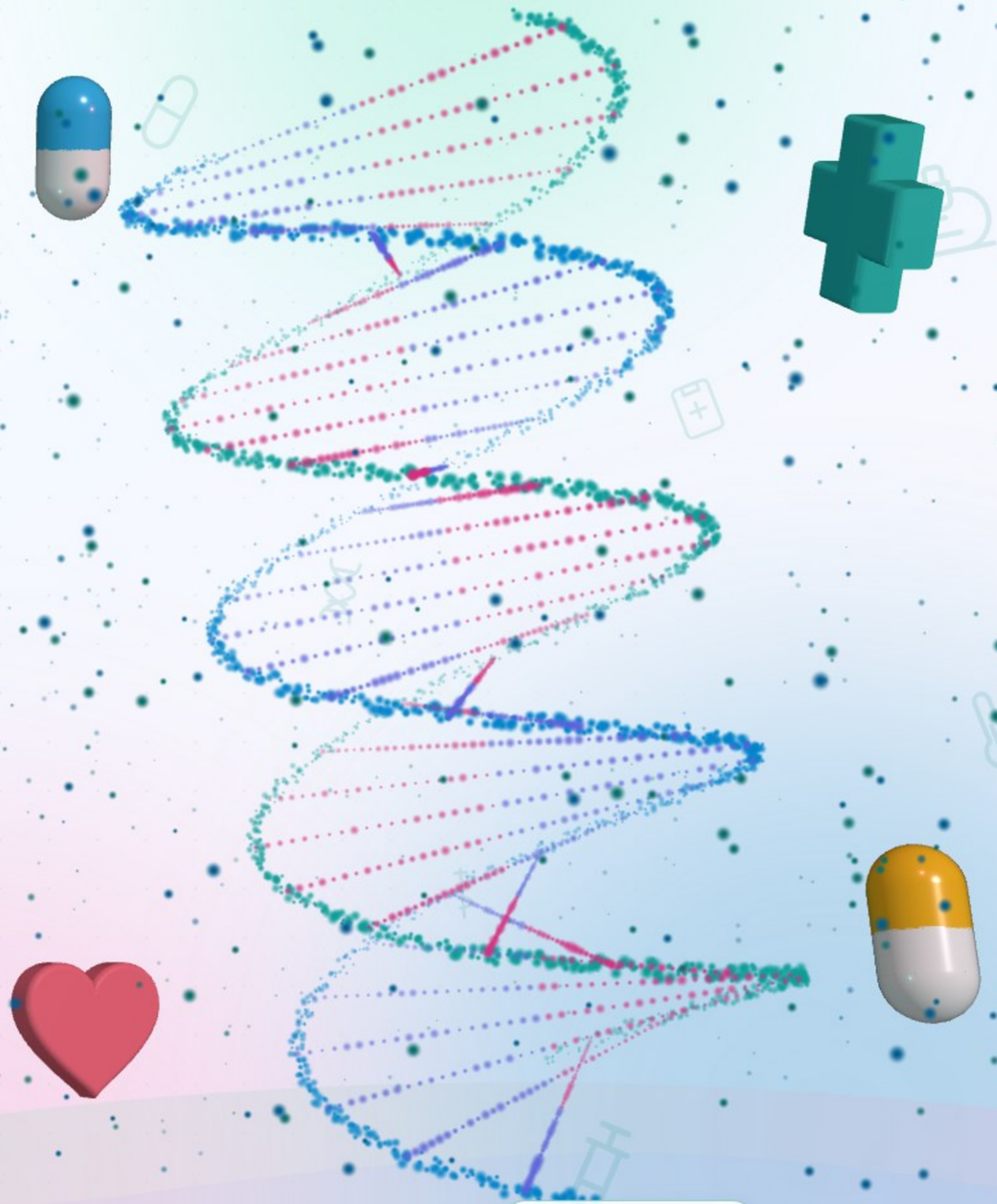
6 microservices + **1** commun

Spring Boot · Spring Cloud

RabbitMQ

Keycloak

Angular



Une équipe, un microservice chacun

EY — **Eya**
Dossier médical

AM — **Amine**
Ordonnances

FE — **Fedia**
Médicaments

AZ — **Aziz**
Pharmacie

TA — **Taha**
Analyses

YA — **Yazid**
Rendez-vous

RÉALISÉ EN COMMUN · TECHNOLOGIE AVANCÉE

user-service · MongoDB

comptes Keycloak · rôles · notifications temps réel (SSE)

Au programme



01 Problème & solution
un dossier médical éclaté, MediVault

02 Acteurs & fonctionnalités
cas d'utilisation et scénario

03 Architecture & données
microservices, Gateway, une base par service

04 Communication & résilience
OpenFeign, RabbitMQ, circuit breaker

05 Organisation & planning
Git, Trello, étapes et risques

06 Nos microservices
une minute par membre

Un dossier médical éclaté

Chaque professionnel garde ses propres informations, souvent sur papier : le patient transporte tout lui-même.

Informations dispersées

Aucun professionnel n'a une vue complète de l'historique du patient.

Erreurs de prescription

Un médicament prescrit malgré une allergie ou une interaction.

Ordonnances papier

La même ordonnance peut être utilisée plusieurs fois ou falsifiée.

Aucun contrôle

Le patient ne décide pas qui consulte ses données médicales.

Comment centraliser le dossier médical du patient de façon sûre, tout en réduisant les erreurs de prescription ?

MediVault : le coffre-fort médical du patient

medi + vault : une plateforme web en microservices, en médecine générale. Données fictives.

01

Un dossier unique

Visible par un professionnel **seulement avec l'accord du patient**. Chaque accès est tracé.

02

Une ordonnance sûre

Allergies et interactions vérifiées avant validation, **délivrée une seule fois** en pharmacie.

03

Des résultats en direct

Le laboratoire envoie les résultats au dossier, le patient est **notifié en temps réel**.

Cinq acteurs



Patient

- Consulte son dossier
- Donne / retire les accès
- Prend rendez-vous
- Reçoit les notifications



Médecin

- Lit le dossier autorisé
- Ajoute une consultation
- Rédige les ordonnances
- Gère son agenda



Pharmacien

- Cherche l'ordonnance par code
- La délivre une seule fois



Laborantin

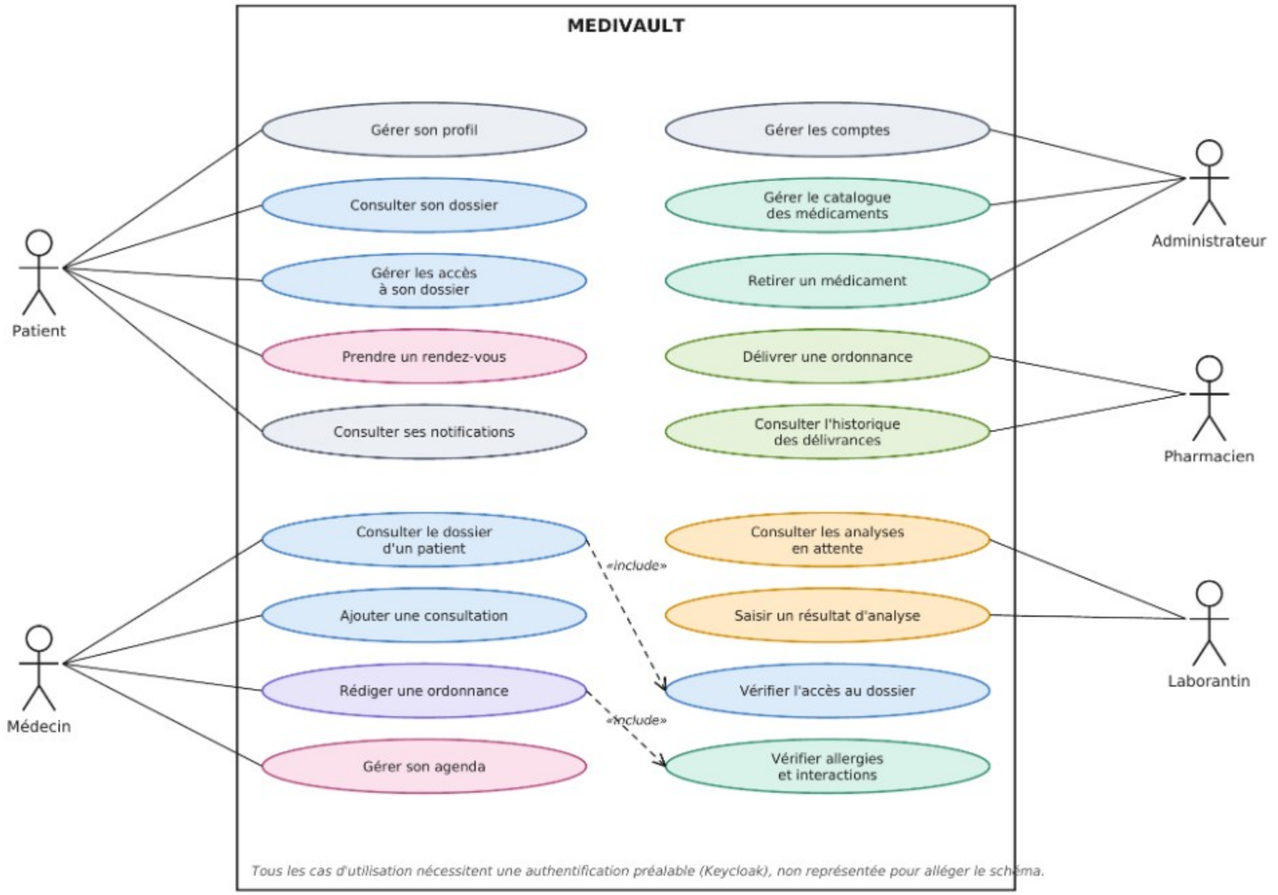
- Voit les analyses à traiter
- Saisit les résultats



Administrateur

- Gère les comptes
- Gère le catalogue
- Retire un médicament

Diagramme de cas d'utilisation



«include» consulter le dossier → vérifier l'accès

«include» rédiger une ordonnance → vérifier allergies et interactions

authentification Keycloak pour tous

Dix fonctionnalités

Chaque fonctionnalité répond à un problème et passe par un ou plusieurs microservices.

01
Inscription & profils
Keycloak, profil pro (adresse, horaires)

02
Dossier unique
créé à l'inscription (UtilisateurCree)

03
Contrôle des accès
le patient accorde / retire, accès ACTIF

04
Rendez-vous
créneaux selon les horaires du pro

05
Consultation
motif, diagnostic, analyses demandées

06
Ordonnance sécurisée
allergie / interaction → 422 + alternative

07
Délivrance unique
totale ou partielle, jamais deux fois

08
Analyses
EN_ATTENTE → EN_COURS → TERMINEE

09
Retrait d'un médicament
bloqué en pharmacie immédiatement

10
Notifications temps réel
MongoDB + Server-Sent Events

Le parcours d'Ahmed

Un scénario de bout en bout, utilisé pour la démonstration. Noms et données fictifs.



1 · Inscription
Ahmed s'inscrit → dossier vide
`user · dossier`

2 · Accès accordé
au Dr Karim
`dossier`

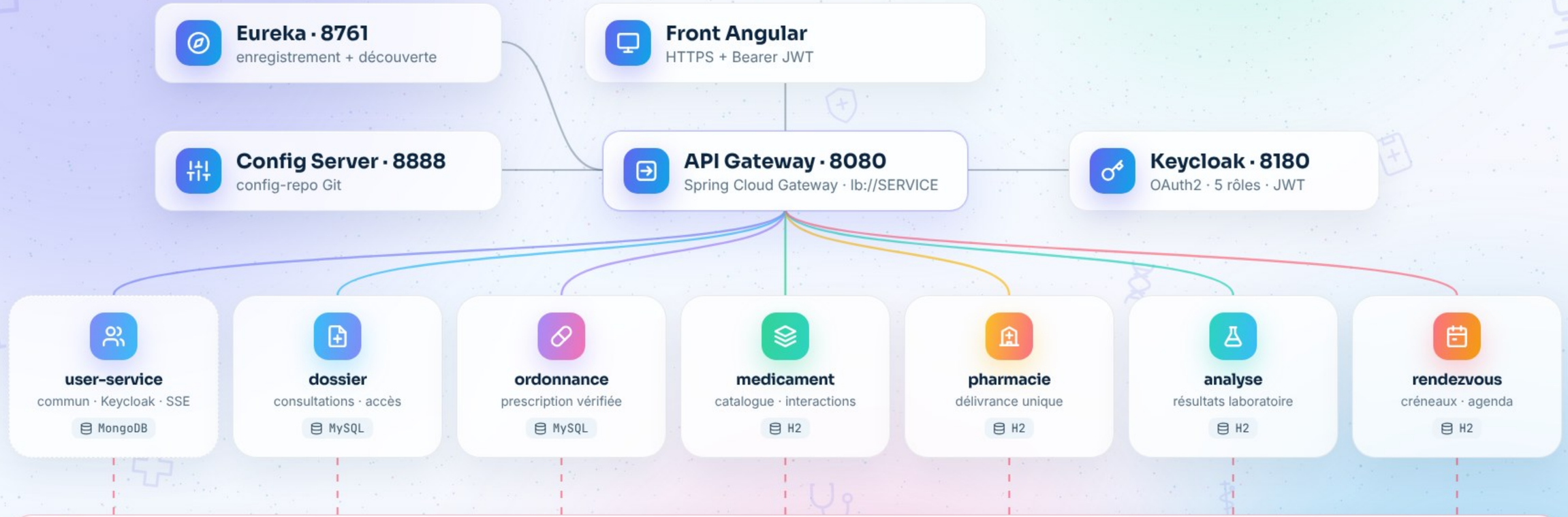
3 · RDV + consultation
angine, bilan sanguin demandé
`rendezvous · dossier`

4 · Ordonnance
Augmentin → 422, alternative validée
`ordonnance · medicament`

5 · Délivrance
une seule fois, totale
`pharmacie`

6 · Résultat BioLab
glycémie 1,30 g/L → ELEVE
ajouté au dossier + notification

Architecture globale



REST via la Gateway | OpenFeign + Resilience4j | RabbitMQ asynchrone | 1 base par service | Docker Compose

Une base par service

Découpage par contexte métier, pas par table. Aucune base partagée.

H2

`medicament · pharmacie · analyse · rendezvous`

Base embarquée et légère pour des données structurées simples. Catalogue rechargé au démarrage par data.sql.

MySQL

`dossier · ordonnance`

Données médicales et légales à conserver durablement : transactions et relations (consultations, lignes d'ordonnance).

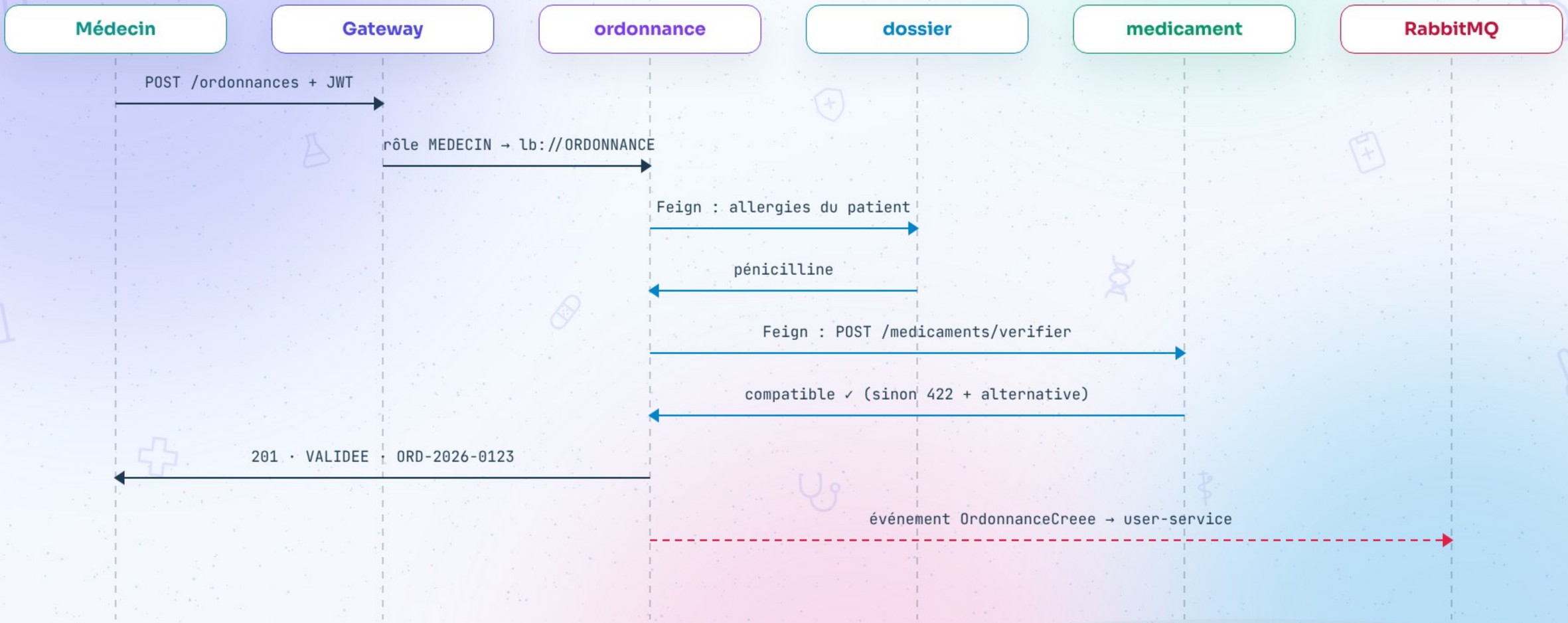
MongoDB

`user-service (commun)`

Champs différents selon le rôle (Professionnel hérite d'Utilisateur), notifications aux champs optionnels, index TTL.

Entre services, on garde seulement l'identifiant : `patient_id`, `ordonnance_id`… jamais de clé étrangère entre bases.

Le médecin prescrit : qui parle à qui ?



Synchrone OpenFeign · réponse immédiate nécessaire

Asynchrone RabbitMQ · le producteur n'attend personne

Et si un service tombe ?

CIRCUIT BREAKER RESILIENCE4J · MÉTHODE DE REPLI

APPEL SYNCHRONE	SI LE SERVICE APPELÉ EST INDISPONIBLE	POURQUOI
`dossier → user`	**Accès refusé (503)**	aucune donnée sans vérification
`ordonnance → dossier / medicament`	**EN_ATTENTE_VERIFICATION, non délivrable**	sécurité du patient d'abord
`medicament → ordonnance`	**Retrait publié quand même**	ne jamais bloquer un retrait
`pharmacie → ordonnance`	**Délivrance refusée (503)**	évite une double délivrance
`analyse → dossier`	**Saisie refusée (503)**	fail-closed
`rendezvous → user`	**Créneaux non générés**	pas de créneau hors horaires

Côté asynchrone

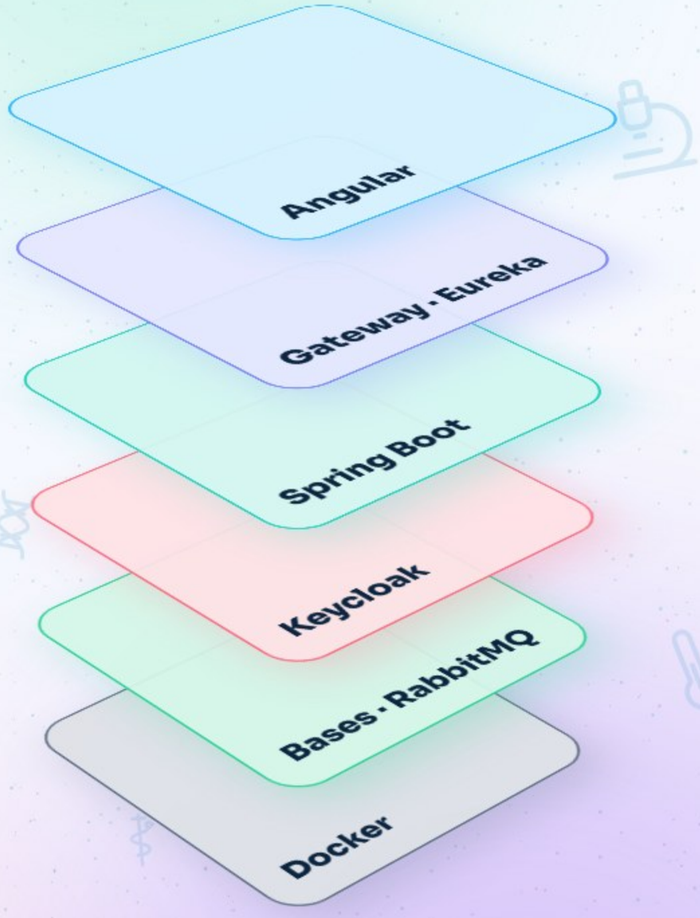
Messages **persistants**, une **file durable** par service consommateur.

Un service arrêté retrouve ses messages dans sa file et les traite **à son redémarrage**.

Acquittement **après** traitement : aucun événement perdu.

La stack technique

FRONT	Angular	TypeScript		
PASSERELLE	Spring Cloud Gateway	Eureka	Config Server	
SERVICES	Spring Boot 3	Spring Data JPA · MongoDB	OpenFeign	Resilience4j
SÉCURITÉ	Keycloak	OAuth2 · JWT · 5 rôles		
DONNÉES & MESSAGES	H2 · MySQL · MongoDB	RabbitMQ		
LIVRAISON & SUIVI	Docker Compose	Swagger	GitHub · Trello	



Qui fait quoi

CHACUN : BACK-END + FRONT-END DE SA GESTION

MEMBRE	MICROSERVICE	PARTIE COMMUNE
Eya	`dossier-service`	Keycloak : realm et rôles
Amine	`ordonnance-service`	RabbitMQ : clés et événements
Fedia	`medicament-service`	dépôt Git, README, diagrammes
Aziz	`pharmacie-service`	Gateway, Eureka, Config Server
Taha	`analyse-service`	Angular, Docker Compose, SSE
Yazid	`rendezvous-service`	présentation, suivi Trello
Tous	`user-service`	MongoDB, notifications

DÉPÔT GIT PUBLIC

```
medivault-4dasi4-keystone/
├── Backend/
│   ├── eureka-server · config-server · api-gateway
│   ├── user-service · dossier-service · ordonnance-service
│   ├── medicament-service · pharmacie-service
│   └── analyse-service · rendezvous-service
├── Frontend/          (Angular)
├── Documentation/
│   ├── Presentation/
│   └── Diagrams/
└── README.md
```

TRELLO

Une carte par tâche, un responsable, une échéance · À faire / En cours / Terminé. Chaque membre commite dans son propre service.

Planning & risques

S1–S3
Conception
sujet, découpage, contrats

S4
Validation 1
conception, dépôt, projets créés

S5
Gateway + CRUD
endpoints, Swagger

S6
Synchrone
OpenFeign, Resilience4j

S7
Asynchrone
RabbitMQ

S8
Sécurité
Keycloak, rôles, JWT

S9
Docker + front
docker-compose, Angular

S10
Soutenance
démonstration finale

PRINCIPAUX RISQUES → MESURE PRÉVUE

Dépendances entre services
contrats d'API figés dès S4, données simulées

Complexité de Keycloak
un référent, rôles simples, mise en place progressive

Intégration tardive
lancement commun chaque semaine

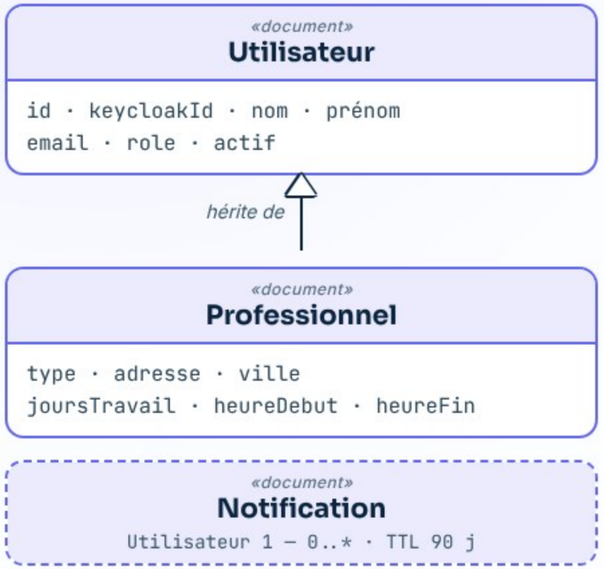
Catalogue trop large
30 médicaments, 10 interactions (data.sql)

Contributions inégales
tâches Trello, commits de chacun

user-service · commun

Microservice avancé : les personnes, leurs droits et leur information en temps réel.

DIAGRAMME DE CLASSES



API REST /api · codes HTTP

POST	/utilisateurs	201·400·409
GET	/utilisateurs/{id}	200·404
GET	/professionnels?type=&ville=	200
GET	/professionnels/{id}/disponibilites	200·404
GET	/notifications/stream (SSE)	200
PUT	/notifications/{id}/lue	200·404

INTERACTIONS

- SYNC → **Keycloak Admin API** : crée le compte et son rôle.
- PUBLIE **UtilisateurCree** → dossier.
- CONSOMME **AccesAccorde, OrdonnanceCreee, ResultatDisponible, RendezVousConfirme** → notification.
- AVANCÉ MongoDB + héritage · SSE temps réel · index TTL.

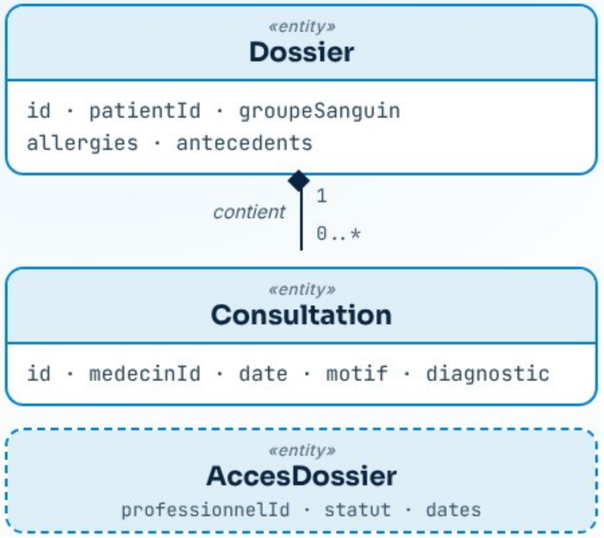
TA Présenté par : **Taha** · base MongoDB · user_db

Dossier médical

Le dossier unique du patient, ses consultations et qui peut le lire.

01

DIAGRAMME DE CLASSES



API REST /api · codes HTTP

GET	/dossiers/patient/{patientId}	200·403·404
POST	/dossiers/{id}/consultations	201·400·403
GET	/dossiers/patient/{patientId}/allergies	200·404
GET	/dossiers/patient/{patientId}/acces	200·404
POST	/dossiers/{id}/acces	201·409
DELETE	/dossiers/{id}/acces/{professionnelId}	204·404

INTERACTIONS

- **FEIGN** → **user** : le professionnel existe et est actif ?
- **PUBLIE** **ConsultationAjoutee**, **AccesAccorde**
- **CONSOMME** **UtilisateurCree** → dossier vide · **ResultatDisponible**, **OrdonnanceDelivree** → historique
- **PANNE** user indisponible → accès refusé **503**.

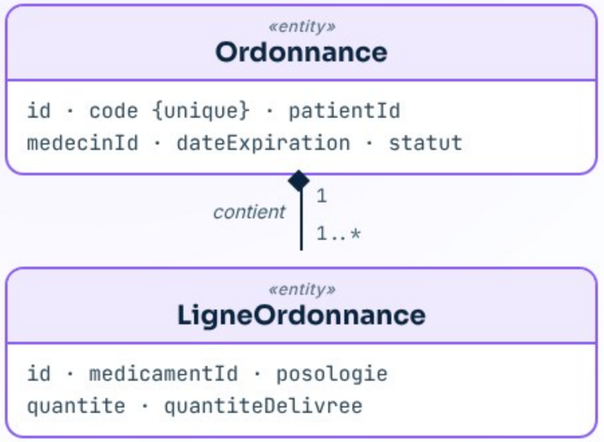
EY Responsable : **Eya** · base MySQL · dossier_db

Ordonnances

Rédaction vérifiée avant validation, puis cycle de vie jusqu'à la délivrance.

02

DIAGRAMME DE CLASSES



API REST /api · codes HTTP

POST	/ordonnances	201·400·422
GET	/ordonnances/{code}	200·404
GET	/ordonnances?patientId=	200
GET	/ordonnances?medicamentId=&statut=	200

INTERACTIONS

- FEIGN → **dossier** : allergies · → **medicament** : /verifier
- PUBLIE **OrdonnanceCreee** → user
- CONSOMME **OrdonnanceDelivree** → statut DELIVREE
- PANNE ordonnance **EN_ATTENTE_VERIFICATION**, non délivrable.

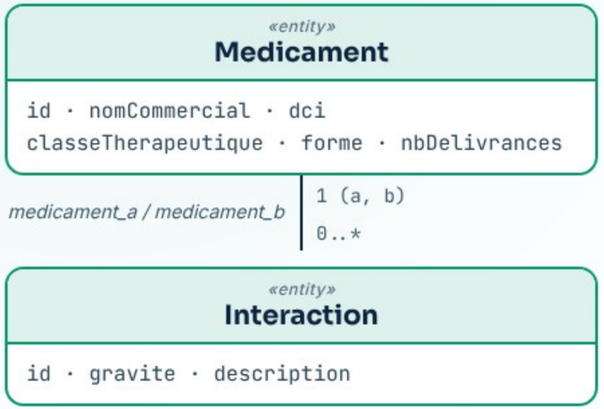
AM Responsable : **Amine** · base MySQL · ordonnance_db

Médicaments

Catalogue, interactions, vérification de compatibilité et retrait du marché.

03

DIAGRAMME DE CLASSES



API REST /api · codes HTTP

GET	/medicaments	200
POST	/medicaments	201·400
POST	/medicaments/verifier	200·400
DELETE	/medicaments/{id}	204·404

INTERACTIONS

- FEIGN → **ordonnance** : ordonnances actives avec ce médicament
- PUBLIE **MedicamentRetire** → pharmacie
- CONSOMME **OrdonnanceDelivree** → nbDelivrances + 1
- PANNE le retrait est **publié quand même**.

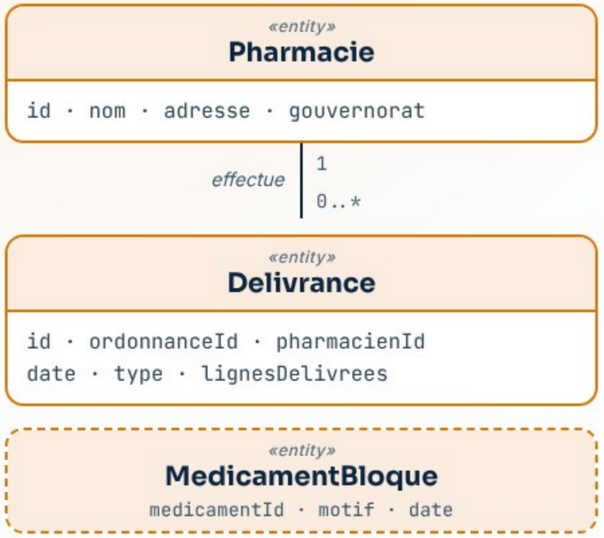
FE Responsable : **Fedia** · base H2 · medicament_db

Pharmacie

Recherche par code et délivrance unique, totale ou partielle.

04

DIAGRAMME DE CLASSES



API REST /api · codes HTTP

GET	/pharmacies	200
POST	/delivrances	201·409·422
GET	/delivrances?pharmacieId=	200
GET	/delivrances/ordonnance/{id}	200·404

INTERACTIONS

- FEIGN → **ordonnance** : valide et non délivrée ?
- PUBLIE **OrdonnanceDelivree** → ordonnance, dossier, medicament
- CONSOMME **MedicamentRetire** → MedicamentBloque
- PANNE délivrance refusée **503** : pas de double délivrance.

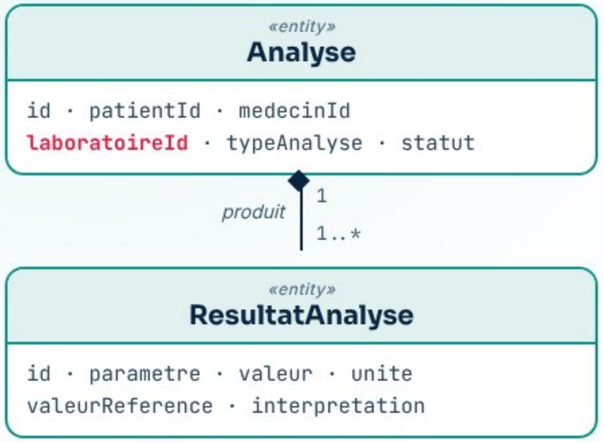
AZ Responsable : **Aziz** · base `H2` · `pharmacie_db`

Analyses

Analyses demandées en consultation, affectées au laboratoire du rendez-vous.

05

DIAGRAMME DE CLASSES



API REST /api · codes HTTP

GET	/analyses?statut=EN_COURS&laboratoireId=	200
POST	/analyses/{id}/resultats	201·400·403·503
GET	/analyses/patient/{patientId}	200
GET	/analyses/{id}	200·404

INTERACTIONS

- **FEIGN** → **dossier** /acces : le laboratoire a-t-il un accès ACTIF ?
- **PUBLIE** **ResultatDisponible** → dossier, user
- **CONSOMME** **ConsultationAjoutee** → EN_ATTENTE · **RendezVousConfirme** → EN_COURS + laboratoireId
- **PANNE** dossier indisponible → saisie refusée **503**.

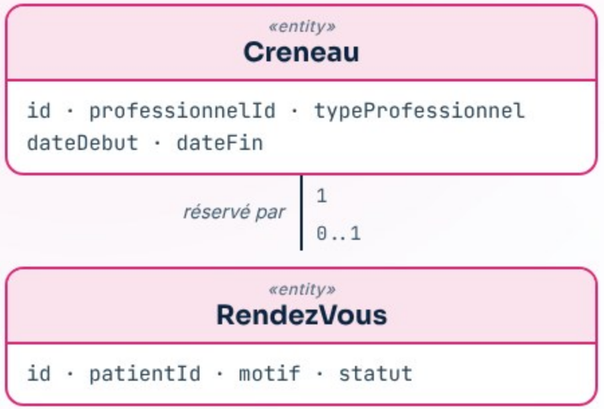
TA Responsable : **Taha** · base `H2` · `analyse_db`

Rendez-vous

Créneaux générés selon les horaires du médecin ou du laboratoire.

06

DIAGRAMME DE CLASSES



API REST /api · codes HTTP

POST	/creneaux/generer?professionnelId=	201·404
GET	/creneaux?professionnelId=&date=	200
POST	/rendezvous	201·409
PUT	/rendezvous/{id}/annuler	200·404

INTERACTIONS

- FEIGN → **user** : jours et heures de travail du professionnel
- PUBLIE **RendezVousConfirme** → user, analyse
- CONSOMME **ConsultationAjoutee** → rendez-vous TERMINE
- PANNE créneaux non générés, « réessayer plus tard ».

YA Responsable : **Yazid** · base H2 · rendezvous_db



MERCI



Merci pour votre attention



Équipe Keystone · nous sommes prêts pour vos questions.



